



THIẾT KẾ LUẬN LÝ SỐ

CHƯƠNG 1: MẠCH TUẦN TỰ



NỘI DUNG

1.1. SR Latch

1.2. Gated SR Latch.

1.3. Gated D Latch

1.4. Các loại Flip-Flop

1.5. Phân tích mạch tuần tự

1.6. Mô hình máy trạng thái hữu hạn

1.7. Tổng hợp mạch tuần tự



1.7. Tổng hợp mạch tuần tự

1.7.1. Tối thiểu hóa trạng thái

1.7.2. Mã hóa trạng thái

1.7.3. Lựa chọn Flip-Flop

1.7.4. Tối ưu hóa và định thời



1.7 Tổng hợp mạch tuần tự

1.7.1. Tối thiểu hóa trạng thái



1.7.1. Tối thiểu hóa trạng thái

- Làm giảm số lượng trạng thái và do đó, số lượng flip-flop cần thiết để thực hiện mạch cũng giảm.
- Dựa trên khái niệm tương đương hành vi trong đó hai trạng thái tương đương nhau nếu chúng tạo ra cùng một chuỗi các ký hiệu trạng thái tiếp theo và đầu ra cho mọi chuỗi ký hiệu đầu vào.
 - hai trạng thái s_j và s_k trong FSM được cho là tương đương nếu hai điều kiện sau là đúng:
 - Điều kiện 1: Cả hai trạng thái s_j và s_k đều tạo ra cùng một ký hiệu đầu ra cho mọi ký hiệu đầu vào i : nghĩa là $h(s_j, i) = h(s_k, i)$
 - Điều kiện 2: Cả hai trạng thái đều có các vị trí tiếp theo tương đương cho mọi ký hiệu đầu vào i : nghĩa là $f(s_j, i) = f(s_k, i)$
- Các bước thực hiện:
 1. Phân vùng trạng thái thành các lớp tương đương
 2. Xây dựng FSM mới với một trạng thái cho mỗi lớp tương đương

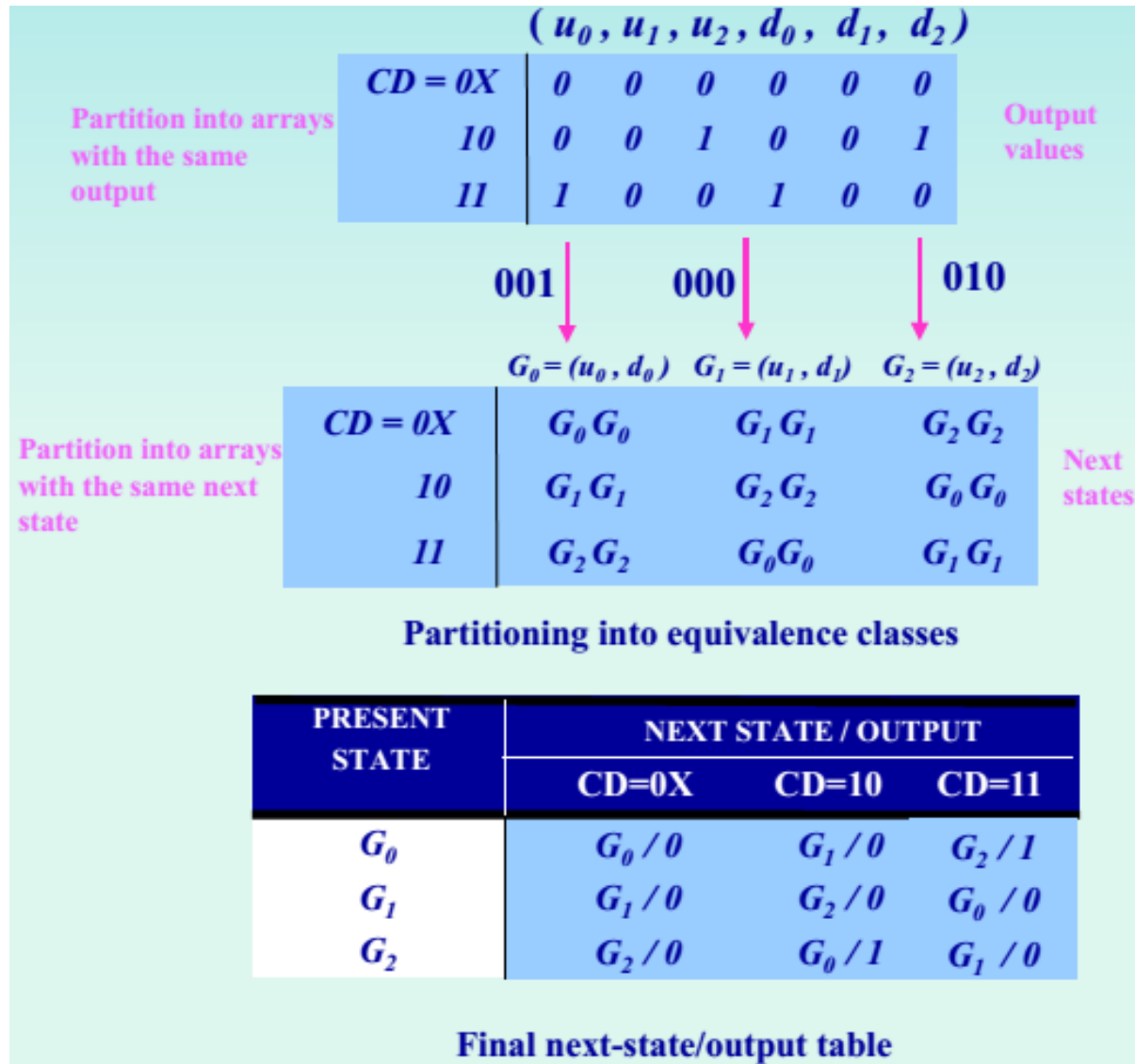
1.7.1. Tối thiểu hóa trạng thái

- Example: State reduction
- Problem: Derive the minimal-state FSM for the modulo-3 counter

PRESENT STATE	NEXT STATE / OUTPUT		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_1 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_1 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$



1.7.1. Tối thiểu hóa trạng thái



PRESENT STATE	NEXT STATE / OUTPUT		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_1 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_1 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$

1.7.1. Tối thiểu hóa trạng thái

s_1						
s_2						
s_3		X				
s_4	X					
s_5		$\langle s_3, s_4 \rangle$		$\langle s_2, s_6 \rangle$ $\langle s_0, s_4 \rangle$		
s_6						
	s_0	s_1	s_2	s_3	s_4	s_5

Implication table

Implication table

- Step 1: Enter x for any pair of states that differ in output values
- Step2: Enter implied equivalent states for every pair of states
- Step3: Enter x for the non equivalent next-state pair
- Step4: Form equivalence classes using transitivity : $s_1 \equiv s_2$ & $s_2 \equiv s_3 \Rightarrow s_1 \equiv s_3$

1.7.1. Tối thiểu hóa trạng thái

- Example: State reductions with implication table.
- Problem: Find the minimal number of states for the FSM specified by the table below.

Equivalence classes:

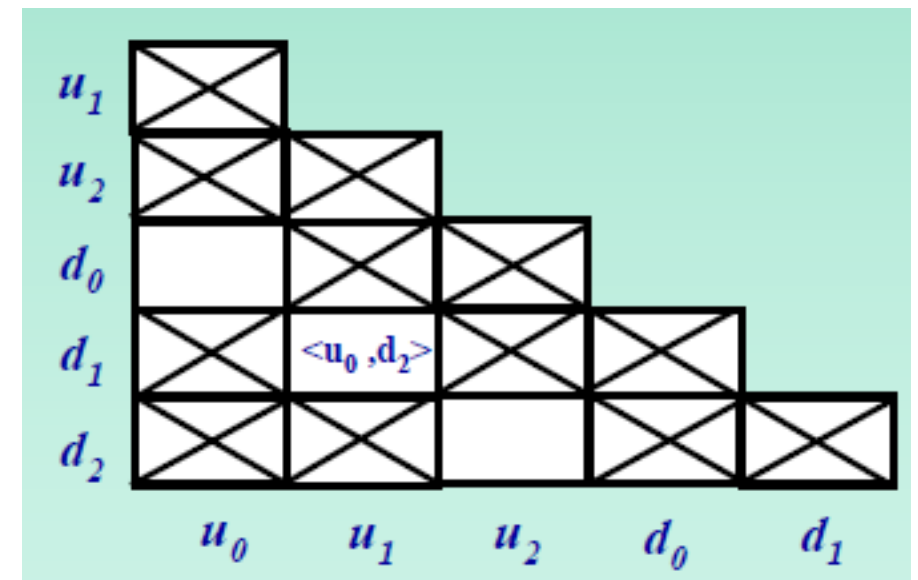
$\langle u_0, d_0 \rangle$

$\langle u_1 \rangle$

$\langle d_1 \rangle$

$\langle u_2, d_2 \rangle$

PRESENT STATE	NEXT STATE		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_0 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_2 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$





1.7 Tổng hợp mạch tuần tự

1.7.2. Mã hóa trạng thái

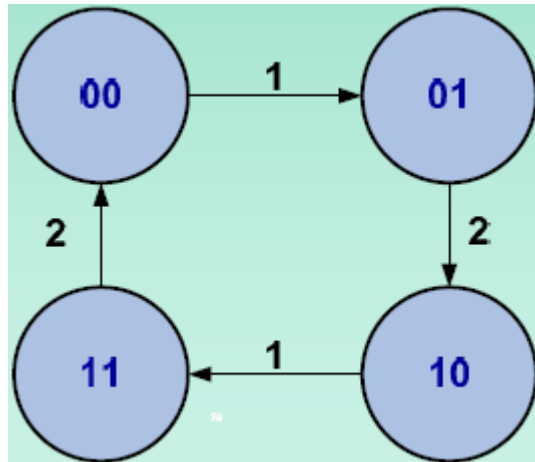
1.7.2. Mã hóa trạng thái

ENCODING NUMBER	s_0	s_1	s_2	s_3
1	00	01	10	11
2	00	01	11	10
3	00	10	01	11
4	00	10	11	01
5	00	11	01	10
6	00	11	10	01
7	01	00	10	11
8	01	00	11	10
9	01	10	00	11
10	01	10	11	00
11	01	11	00	10
12	01	11	10	00
13	10	00	01	11
14	10	00	11	01
15	10	01	00	11
16	10	01	11	00
17	10	11	00	01
18	10	11	01	00
19	11	00	01	10
20	11	00	10	01
21	11	01	00	10
22	11	01	10	00
23	11	10	00	01
24	11	10	01	00

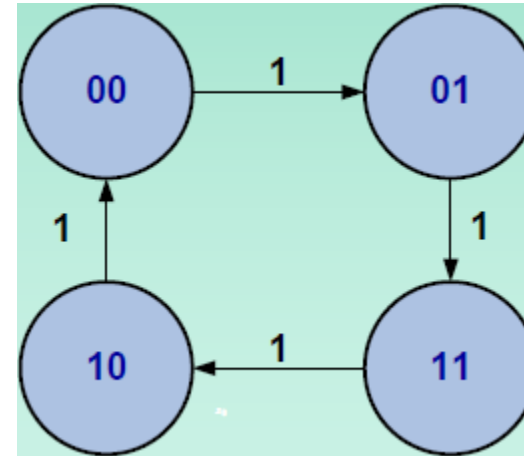
- Chi phí và độ trễ của việc triển khai FSM phụ thuộc vào việc mã hóa các trạng thái tượng trưng
- Ví dụ: bốn trạng thái có thể được mã hóa theo $4!=24$ cách khác nhau.
- Có nhiều hơn $n!$ mã hóa khác nhau cho n trạng thái. Việc khám phá tất cả các bảng mã là không thể. Vì vậy, chúng tôi sử dụng heuristic.
- Ba phương pháp phỏng đoán khác nhau:
 - minimum-bit-change
 - prioritized adjacency
 - one-hot encoding

1.7.2. Mã hóa trạng thái

- Minimum-bit-change
 - gán mã cho các trạng thái sao cho tổng số bit thay đổi cho tất cả các chuyển đổi trạng thái được giảm thiểu.



Straightforward encoding
(total weight is 6)



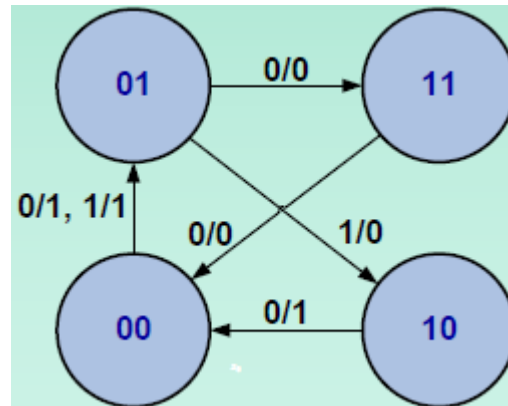
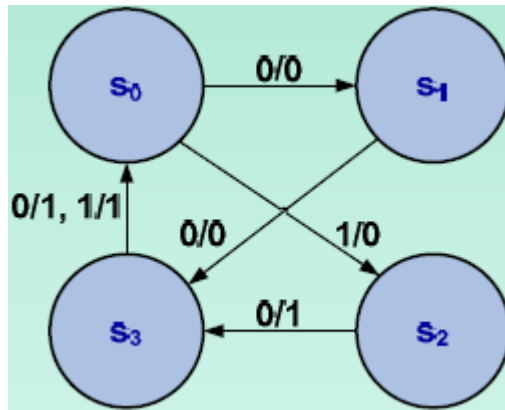
Minimum-bit-change encoding
(total weight is 4)

1.7.2. Mã hóa trạng thái

- Mã hóa theo ưu tiên liên kề
 - Sử dụng số bit trạng thái tối thiểu để mã hóa trạng thái
- Ưu tiên:
 1. Mã hóa liên kề các trạng thái có chung trạng thái kế tiếp ứng với một giá trị ngõ vào
 2. Mã hóa liên kề các trạng thái kế tiếp của cùng một trạng thái
 3. Mã hóa liên kề các trạng thái có chung giá trị ngõ ra với cùng một giá trị ngõ vào

1.7.2. Mã hóa trạng thái

- Mã hóa theo ưu tiên liên kề



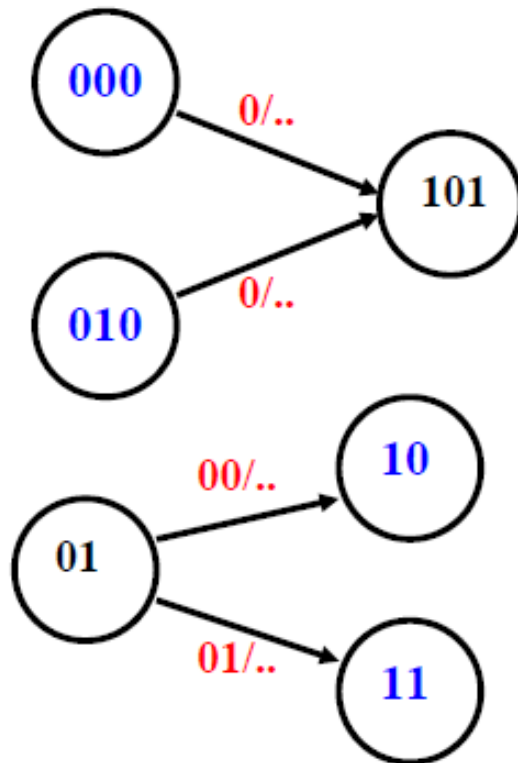
Ưu tiên 1: (s1, s2)

Ưu tiên 2: (s1, s2)

Ưu tiên 3: (s0, s1), (s2, s3)

1.7.2. Mã hóa trạng thái

- Mã hóa theo ưu tiên liên kề



Điều kiện ưu tiên cao nhất

Điều kiện ưu tiên thứ hai



1.7.2. Mã hóa trạng thái

- Mã hóa Hot-one
 - Sử dụng một bit trạng thái để gán cho một trạng thái, nói cách khác, mỗi trạng thái được phân biệt với các trạng thái nhờ bit trạng thái dành cho nó có giá trị 1 trong khi các bit trạng thái còn lại bằng 0
 - Chi phí lớn đối với các FSM có số lượng trạng thái lớn. Nó chỉ được sử dụng trong các FSM nhỏ.



1.7.2. Mã hóa trạng thái

- Ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

1.7.2. Mã hóa trạng thái

- Ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

STATE	ENCODING A	ENCODING B	ENCODING C
	Q_1Q_0	Q_1Q_0	$Q_2Q_1Q_0$
S_0	0 0	0 1	0 0 1
S_1	0 1	0 0	0 1 0
S_2	1 0	1 0	1 0 0

Encoding A = Minimum-bit-change

Encoding B = Prioritized adjacency

Encoding C = Hot-one encoding



1.7.2. Mã hóa trạng thái

- Minimum-bit-change: Ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

STATE	ENCODING A Q_1Q_0
S_0	0 0
S_1	0 1
S_2	1 0

Q_1Q_0 \ CD	CD			
	00	01	11	10
00	0 0	0 0	1 0	0 1
01	0 1	0 1	0 0	1 0
11	X X	X X	X X	X X
10	1 0	1 0	0 1	0 0

$Q_1(next), Q_0(next)$

$$Q_0(next) = Q_0C' + Q_1CD + Q_1'Q_0'CD'$$

$$Q_1(next) = Q_1C' + Q_0CD' + Q_1'Q_0'CD$$

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

Q_1Q_0 \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	0	0	0	0
11	X	X	X	X
10	0	0	0	1

Y

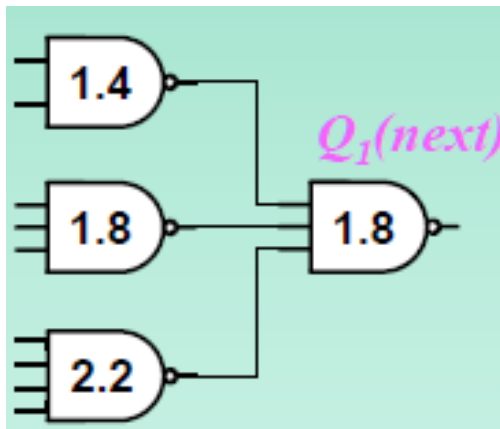
$$Y = Q_1CD' + Q_1'Q_0'CD$$

1.7.2. Mã hóa trạng thái

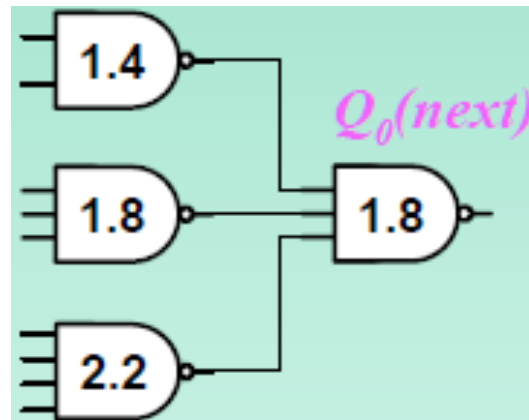
- Minimum-bit-change: ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

$$Q_1(next) = Q_1C' + Q_0CD' + Q_1'Q_0'CD$$

$$Q_0(next) = Q_0C' + Q_1CD + Q_1'Q_0'CD'$$

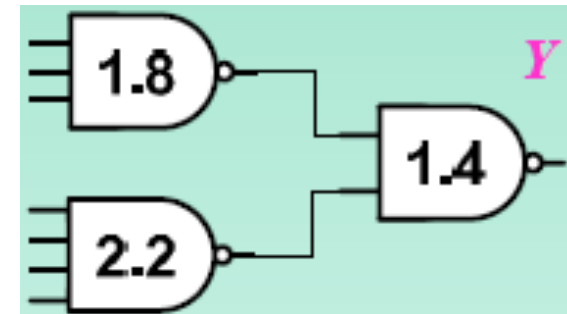


Cost (Q1) = 24
Delay (Q1) = 4.0



Cost (Q0) = 24
Delay (Q0) = 4.0

$$Y = Q_1CD' + Q_1'Q_0'CD$$



Total cost = 66
Max. input delay = 4.0ns
Max. output delay = 3.6ns

1.7.2. Mã hóa trạng thái

- Prioritized adjacency: ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

STATE	ENCODING B Q_1Q_0
S_0	0 1
S_1	0 0
S_2	1 0

Present State	NEXT STATE		
	CD = 0X	CD = 10	CD = 11
S_0	$S_0/0$	$S_1/0$	$S_2/1$
S_1	$S_1/0$	$S_2/0$	$S_0/0$
S_2	$S_2/0$	$S_0/1$	$S_1/0$

Q_1Q_0	CD			
	00	01	11	10
00	0 0	0 0	0 1	1 0
01	0 1	0 1	1 0	0 0
11	X X	X X	X X	X X
10	1 0	1 0	0 0	0 1

$Q_1(next), Q_0(next)$

Q_1Q_0	CD			
	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	0	0	0	1

Y

$$Q_0(next) = Q_0C' + Q_1CD' + Q_1'Q_0'CD$$

$$Q_1(next) = Q_1C' + Q_0CD + Q_1'Q_0'C'D'$$

$$Y = Q_0CD + Q_1CD'$$

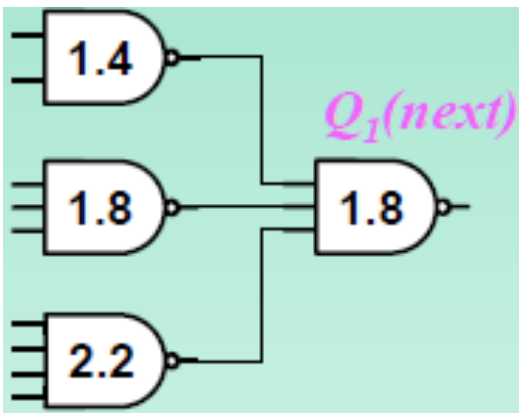
1.7.2. Mã hóa trạng thái

- Prioritized adjacency: ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

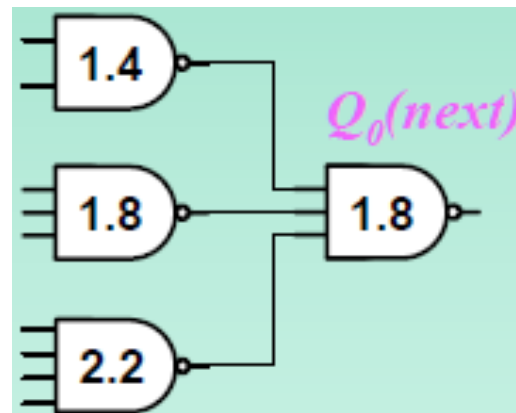
$$Q_0(\text{next}) = Q_0C' + Q_1CD' + Q_1'Q_0'CD$$

$$Q_1(\text{next}) = Q_1C' + Q_0CD + Q_1'Q_0'C'D'$$

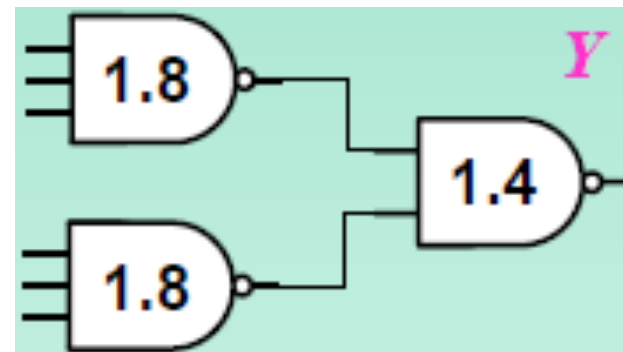
$$Y = Q_0CD + Q_1CD'$$



Cost (Q1) = 24
Delay (Q1) = 4.0



Cost (Q0) = 24
Delay (Q0) = 4.0



Cost (Y) = 16
Delay (Y) = 3.2

Total cost = 64

Max. input delay = 4.0ns

Max. output delay = 3.2ns

1.7.2. Mã hóa trạng thái

- Hot-one encoding : ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

$Q_2Q_1Q_0$		CD			
		00	01	11	10
000		X	X	X	X
001		0	0	1	0
011		X	X	X	X
010		0	1	0	0
100		1	0	0	1
101		X	X	X	X
111		X	X	X	X
110		X	X	X	X

$Q_2(next), Q_1(next), Q_0(next)$

$Q_2Q_1Q_0$		CD			
		00	01	11	10
000		X	X	X	X
001		0	0	1	0
011		X	X	X	X
010		0	0	0	0
100		0	0	0	1
101		X	X	X	X
111		X	X	X	X
110		X	X	X	X

Y

$$Q_2(next) = Q_2C' + Q_0CD + Q_1CD'$$

$$Q_1(next) = Q_1C' + Q_2CD + Q_0CD'$$

$$Q_0(next) = Q_0C' + Q_2CD' + Q_1CD$$

$$Y = Q_0CD + Q_2CD'$$

1.7.2. Mã hóa trạng thái

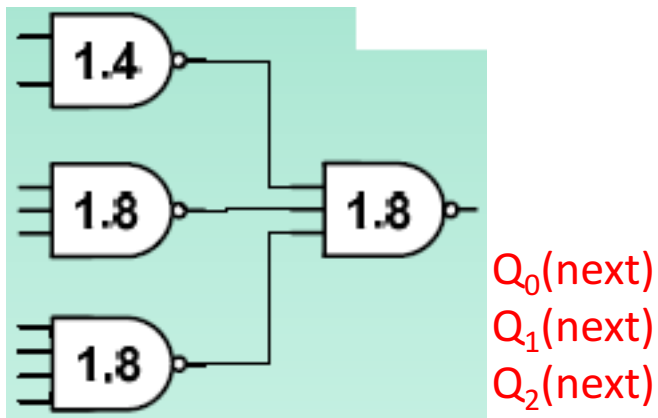
- Hot-one encoding : ví dụ mã hóa trạng thái cho bộ đếm modulo-3.

$$Q_2(\text{next}) = Q_2C' + Q_0CD + Q_1CD'$$

$$Q_1(\text{next}) = Q_1C' + Q_2CD + Q_0CD'$$

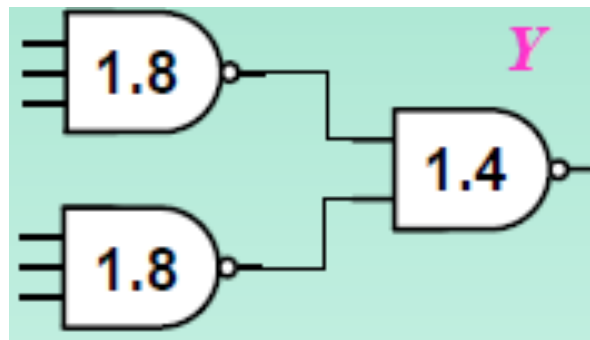
$$Q_0(\text{next}) = Q_0C' + Q_2CD' + Q_1CD$$

$$Y = Q_0CD + Q_2CD'$$



$$\text{Cost } (Q_0) = \text{Cost } (Q_1) = \text{Cost } (Q_2) = 22$$

$$\text{Delay } (Q_0) = \text{Delay } (Q_1) = \text{Delay } (Q_2) = 3.6$$



$$\text{Cost } (Y) = 16$$

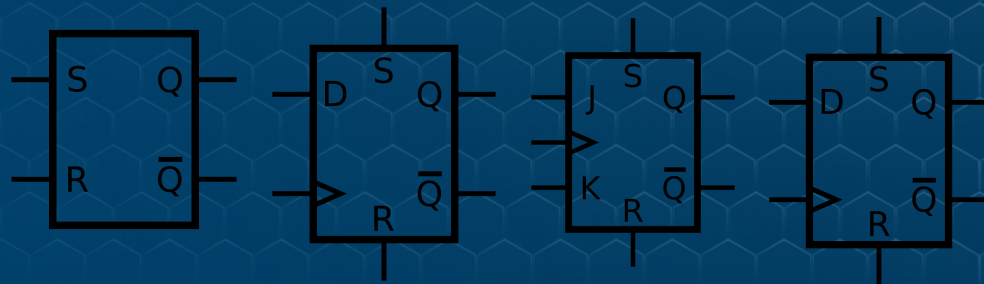
$$\text{Delay } (Y) = 3.2$$

Total cost = 82
 Max. input delay = 3.6ns
 Max. output delay = 3.2ns



1.7. Tổng hợp mạch tuần tự

1.7.3. Lựa chọn Flip-Flop





1.7.3. Lựa chọn Flip-Flop

- Chọn loại flip-flop từ: D, SR, JK, T
- SR và JK giảm chi phí logic đầu vào; nhưng yêu cầu số lượng kết nối gấp đôi.
- D và T yêu cầu kết nối đơn cho mỗi flip-flop.
- T phù hợp với các hàm đếm vì nó thay đổi giá trị đầu ra khi đầu vào bằng 1.



1.7.3. Lựa chọn Flip-Flop

- Thiết kế bộ đếm Modulo-3 với các flip-flop khác nhau, sử dụng mã hóa Minimum-bit-change

Q ₁ Q ₀ \ CD	CD			
	00	01	11	10
00	00	00	10	01
01	01	01	00	10
11	XX	XX	XX	XX
10	10	10	01	00

<i>Q (present)</i>	<i>Q (next)</i>	S R	J K	T	D
0	0	0 X	0 X	0	0
0	1	1 0	1 X	1	1
1	0	0 1	X 1	1	0
1	1	X 0	X 0	0	1

1.7.3. Lựa chọn Flip-Flop

➤ Thiết kế bộ đếm Modulo-3 sử dụng SR flip-flop, mã hóa Minimum-bit-change

Q ₁ Q ₀ \ CD		CD			
		00	01	11	10
00		0 X 0 X	0 X 0 X	1 0 0 X	0 X 1 0
01		0 X X 0	0 X X 0	0 X 0 1	1 0 0 1
11		X X X X	X X X X	X X X X	X X X X
10		X 0 0 X	X 0 0 X	0 1 1 0	0 1 0 X

S₁, R₁, S₀, R₀

$$S_1 = Q_0CD' + Q_1'Q_0'CD \quad (\text{cost} = 18, \text{delay} = 3.6)$$

$$R_1 = Q_1C = (Q_1' + C')' \quad (\text{cost} = 4, \text{delay} = 1.4)$$

$$S_0 = Q_1CD + Q_1'Q_0'CD' \quad (\text{cost} = 18, \text{delay} = 3.6)$$

$$R_0 = Q_0C = (Q_0' + C')' \quad (\text{cost} = 4, \text{delay} = 1.4)$$

1.7.3. Lựa chọn Flip-Flop

➤ Thiết kế bộ đếm Modulo-3 sử dụng JK flip-flop, mã hóa Minimum-bit-change

CD Q ₁ Q ₀		00				01				11				10			
		00		01		10		11		00		01		10		11	
00		0	X	0	X	0	X	0	X	1	X	0	X	0	X	1	X
01		0	X	X	0	0	X	X	0	0	X	X	1	1	X	X	1
11		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
10		X	0	0	X	X	0	0	X	X	1	1	X	X	1	0	X

J₁, K₁, J₀, K₀

$$J_1 = Q_0CD' + Q_0'CD = (C' + Q_0D + Q_0D')' \quad (\text{cost} = 12, \text{delay} = 2.4)$$

$$K_1 = C \quad (\text{cost} = 0, \text{delay} = 0)$$

$$J_0 = Q_1CD + Q_1'CD' = (C' + Q_1D' + Q_1D')' \quad (\text{cost} = 12, \text{delay} = 2.4)$$

$$K_0 = C \quad (\text{cost} = 0, \text{delay} = 0)$$

1.7.3. Lựa chọn Flip-Flop

➤ Thiết kế bộ đếm Modulo-3 sử dụng JK flip-flop, mã hóa Minimum-bit-change

CD Q ₁ Q ₀		00				01				11				10			
		0		X		0		X		0		X		0		X	
00		0	X	0	X	0	X	0	X	1	X	0	X	0	X	1	X
01		0	X	X	0	0	X	X	0	0	X	X	1	1	X	X	1
11		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
10		X	0	0	X	X	0	0	X	X	1	1	X	X	1	0	X

J₁, K₁, J₀, K₀

$$J_1 = Q_0CD' + Q_0'CD = (C' + Q_0D + Q_0D')' \quad (\text{cost} = 12, \text{delay} = 2.4)$$

$$K_1 = C \quad (\text{cost} = 0, \text{delay} = 0)$$

$$J_0 = Q_1CD + Q_1'CD' = (C' + Q_1D' + Q_1D')' \quad (\text{cost} = 12, \text{delay} = 2.4)$$

$$K_0 = C \quad (\text{cost} = 0, \text{delay} = 0)$$

1.7.3. Lựa chọn Flip-Flop

➤ Thiết kế bộ đếm Modulo-3 sử dụng T flip-flop, mã hóa Minimum-bit-change

Q ₁ Q ₀ \ CD		CD			
		00	01	11	10
00	0	0	0	0	1
01	0	0	0	1	1
11	X	X	X	X	X
10	0	0	0	1	0

T₁, T₀

$$T_1 = Q_1C + Q_0'CD + Q_0CD' \quad (\text{cost} = 22, \text{delay} = 3.6)$$

$$T_0 = Q_0C + Q_1CD + Q_1'CD' \quad (\text{cost} = 22, \text{delay} = 3.6)$$

1.7.3. Lựa chọn Flip-Flop

➤ Thiết kế bộ đếm Modulo-3 sử dụng D flip-flop, mã hóa Minimum-bit-change

		CD			
		00	01	11	10
Q ₁ Q ₀	00	0 0	0 0	1 0	0 1
	01	0 1	0 1	0 0	1 0
	11	X X	X X	X X	X X
	10	1 0	1 0	0 1	0 0

D₁, D₀

$$D_1 = Q_1C' + Q_0CD' + Q_1'Q_0'CD \quad (\text{cost} = 24, \text{delay} = 4.0)$$

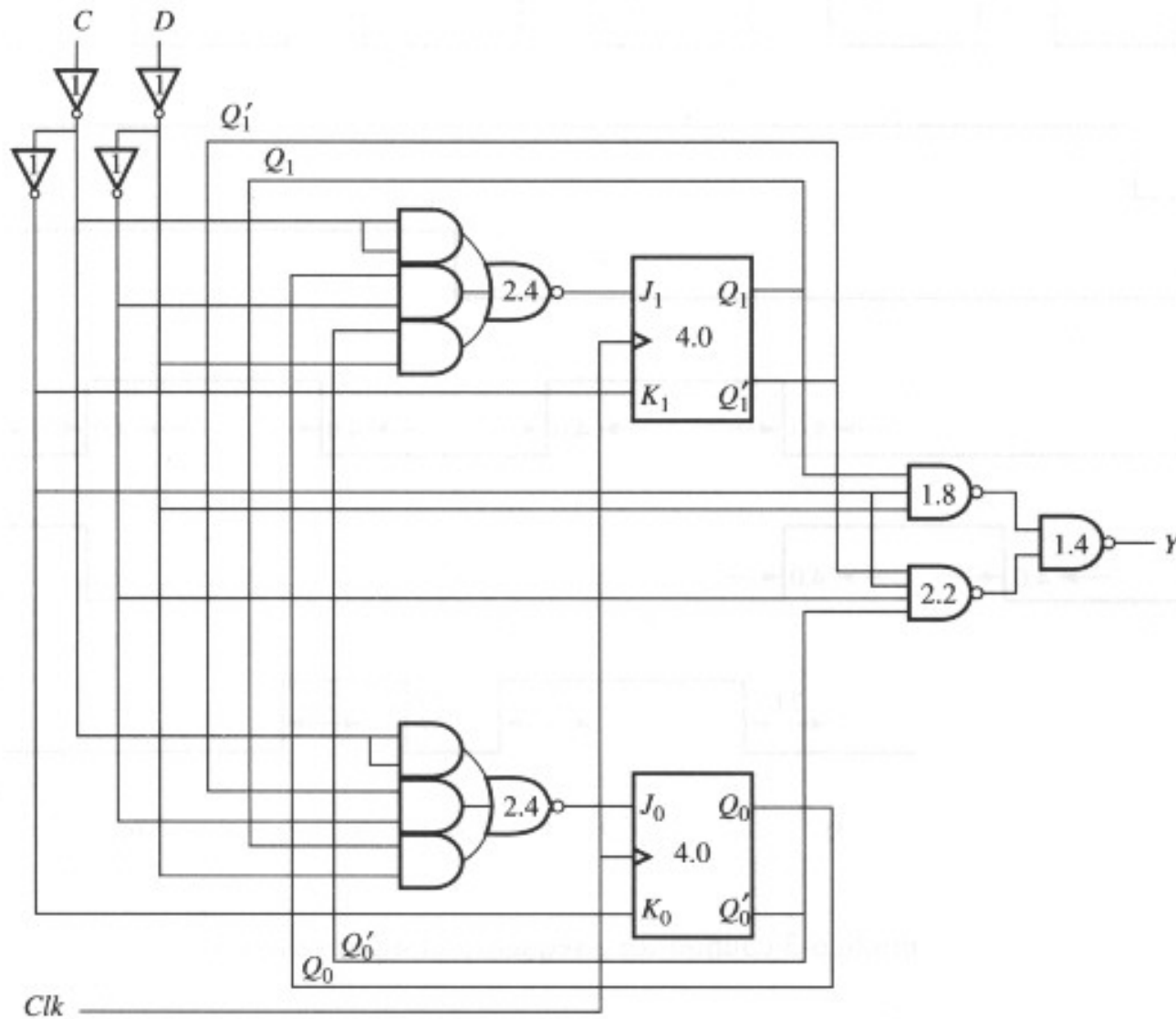
$$D_0 = Q_0C' + Q_1CD + Q_1'Q_0'CD' \quad (\text{cost} = 24, \text{delay} = 4.0)$$



1.7. Tổng hợp mạch tuần tự

1.7.4. Tối ưu hóa và định thời

1.7.4. Tối ưu hóa và định thời



$$J_1 = (C' + Q_0D + Q_0D')$$

$$K_1 = C$$

$$J_0 = (C' + Q_1D' + Q_1D)$$

$$K_0 = C$$

$$Y = Q_1CD' + Q_1'Q_0'CD$$

Input-output delays

C, D to Clk	5.4
Clk to Y	7.6
C, D to Y	5.6
Clk to Clk	7.4

1.7.4. Tối ưu hóa và định thời

